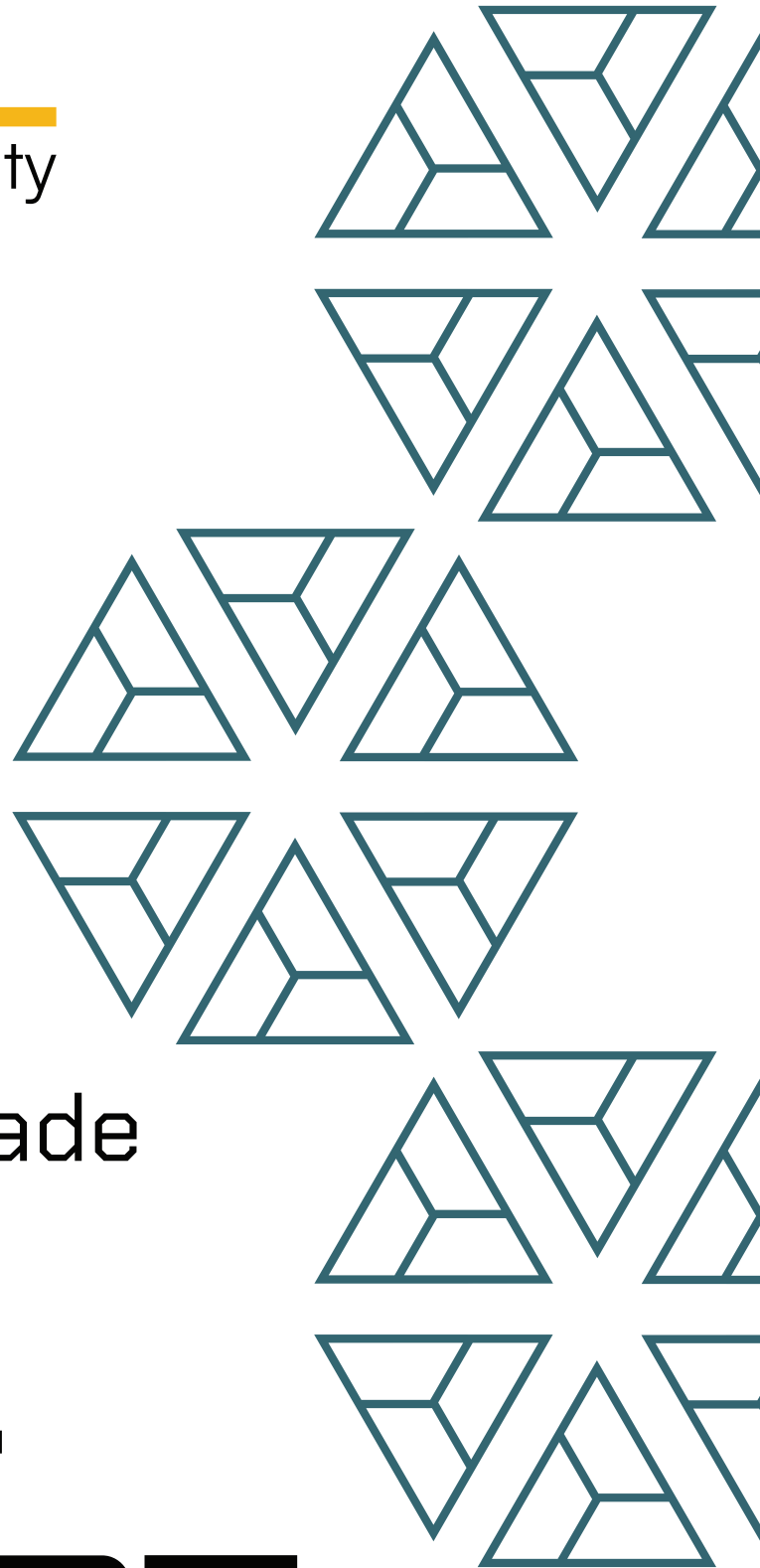




BAIL
security



Ox
SafeGuard Upgrade

FINAL REPORT

May '2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:

Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Ox - Safeguard Upgrade- Audit Report
Website	Ox.org/
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/OxProject/Ox-settler/blob/440478f2cfabc97b2d94b770ce8b72f8c0eedc34/src/deployer/SafeGuard.sol
Resolution 1	https://github.com/OxProject/Ox-settler/blob/0d4ffe7d744964ca253e7de5f84823f00313c5a0/src/deployer/SafeGuard.sol
Resolution 2	https://github.com/OxProject/Ox-settler/blob/8eb50d72f0cf0cc3201a0b8c9dfa4dbfd553a994/src/deployer/SafeGuard.sol

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High						
Medium	5	2		3		
Low	7	3		4		
Informational	9	1		8		
Governance						
Total	21	6		15		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

SafeGuard

The Ox SafeGuard is a Gnosis Safe guard that adds a recovery and delay layer to a Safe. At a high level, it changes the Safe from a standard threshold-controlled wallet into a system where non-unanimous transactions must be queued and pass through a timelock, unanimous transactions can always execute immediately, and lockdown / recovery flows are available if a signer believes there has been a compromise.

Appendix: Transaction queuing

Unless a Safe transaction is approved unanimously by all owners, it must first be queued through the SafeGuard's timelock flow. The entry point for this is `enqueue()`, which is permissionless but still requires valid Safe signatures for the transaction being queued.

When a transaction is queued, the SafeGuard computes the Safe transaction hash and stores a `timelockEnd` timestamp equal to the current timestamp plus the configured delay. The delay value is configurable by the Safe itself, but changing the delay only affects future `enqueue` operations.

Appendix: Transaction Calldata Checking

While the SafeGuard is installed as the Safe's guard, Safe transactions are routed through `checkTransaction()` before execution and `checkAfterExecution()` after execution. The pre-execution checks in `checkTransaction()` inspect the transaction's calldata and reject transactions that would bypass the guard's intended control flow or safety checks. For example, the SafeGuard rejects transactions that attempt to call `checkTransaction()` or `checkAfterExecution()` directly on the guard, since unexpected calls to these functions could corrupt the reentrancy guards or cause other issues. Another example is when the Safe is calling `unlock()` on the SafeGuard. Unlocking is treated as a special recovery action that must be approved unanimously by all Safe owners, and this check is handled by `checkTransaction()` when it detects an unlock call.

When the Safe makes a `delegatecall` rather than a normal call, this is also handled as a special case in `checkTransaction()`. Arbitrary `delegate` calls are disallowed because they could modify the Safe's storage in ways that are impossible for the guard to reason about. The only permitted `delegatecall` target is the `MultiSendCallOnly` contract, which allows the Safe to

batch multiple ordinary calls together. In this scenario, the SafeGuard still decodes the payload and inspects each underlying sub-call.

For MultiSendCallOnly batches, the SafeGuard also requires every sub-call to be followed by a call to `check()` on the SafeGuard. This interleaving forces the multicall to re-check the Safe's invariants between each meaningful action in the batch, rather than only at the very end. The `check()` function verifies that the expected Safe implementation is still in use, no modules have been installed, the fallback handler remains as expected, and the guard has not been removed.

Appendix: Lockdown and unlock

One of the main features of the SafeGuard is the `lockDown()` / `unlock()` flow. The purpose of lockdown is to give any individual owner a way to immediately stop normal Safe execution if they believe there has been a key compromise or another signer issue. Once the SafeGuard is locked down, ordinary queued or threshold-approved transactions can no longer proceed.

The only way to move the Safe out of lockdown is for the Safe itself to call `unlock()`. The SafeGuard requires this unlock transaction to be approved unanimously by all owners.

Appendix: Cancel

The `cancel()` function allows any individual owner to cancel a queued transaction. Only transactions that are still waiting for their timelock to expire can be canceled.

When a transaction is canceled, the SafeGuard sets its `timelockEnd` value to `type(uint256).max`, permanently marking that transaction hash as canceled. Since `enqueue()` rejects transaction hashes that already have a non-zero `timelockEnd`, the same transaction hash cannot be queued again after cancellation.

Appendix: Anti-griefing checks

The `cancel()` and `lockDown()` functions are intentionally callable by any individual Safe owner. While this is important for the system, it also creates a griefing risk: a malicious owner could otherwise use these emergency actions to repeatedly block Safe progress or place the Safe into lockdown.

To address this, the SafeGuard ties both flows to pre-approved recovery transactions. The general principle is that an owner may interrupt normal Safe operation only if they have already committed to a transaction that lets the remaining owners recover.

- `cancel()` requires the caller to have pre-approved their own resignation transaction. If an owner repeatedly cancels queued transactions to prevent the Safe from making progress, the other owners can use the pre-approved resignation to remove them from the Safe.
- `lockDown()` requires the caller to have pre-approved an `unlock()` transaction. This prevents an owner from putting the Safe into lockdown without also committing to a transaction that can bring the Safe back out of lockdown.
- If the locking owner has already pre-approved a resignation at the current nonce, `lockDown()` requires another resignation approval at the next nonce. This prevents the owner from using lockdown to invalidate the earlier resignation approval: because `unlock()` is expected to consume the current nonce, the resignation used after `unlock` must be valid at the following nonce.
- `cancel()` cannot be used to cancel the caller's own resignation transaction.

Together, these checks preserve the intended tradeoff: a single owner can quickly stop suspicious activity, but only if the other owners still have a clear path to recover and get the Safe working again.

Appendix: Post Transaction Checks

After each guarded transaction, the SafeGuard checks that the Safe still satisfies the core assumptions the guard relies on. In particular, it verifies that the Safe is still using the expected implementation, no modules have been installed, the fallback handler is still the expected one, and the SafeGuard itself has not been made an owner.

The SafeGuard also checks that the Safe still has a valid owner / threshold configuration. This includes a minimum owner count, currently 4, a minimum threshold, currently 2, and enough owners outside the threshold for the recovery flows to remain meaningful, which is also implicitly 2. If any of these post-transaction invariants are broken, the transaction reverts.

Appendix: Deployment checks

The SafeGuard performs several deployment-time checks to make sure it is being attached to the type of Safe deployment it was designed to protect. At a high level, these checks confirm that the target Safe is a supported Safe proxy, that the deployments (Safe singleton, fallback handler, and MultiSendCallOnly) are coming from the expected factory, and that the guard was compiled for the expected EVM version.

The SafeGuard.sol file contains separate guard variants for Safe 1.3.0 and Safe 1.4.1, and additional variants for zkSync Era. The 1.3.0 deployment path performs constructor checks and Safe invariant checks during deployment. If those checks fail, the guard marks itself as removed rather than risking a partially installed guard that could brick the Safe. For Safe 1.4.1, the guard is deployed before being enabled, so validation is tied to the ERC165 `supportsInterface()` check that the Safe performs when enabling a guard.

Appendix: zkSync Era contracts

The zkSync Era variants exist because deterministic deployment and bytecode identification work differently on Era than they do on a typical EVM chain. The core SafeGuard logic is the same, but the Era contracts override the deployment checks and address-derivation logic to account for the differences.

Privileged Functions

- `checkTransaction`
- `checkAfterExecution`
- `setDelay`
- `cancel`
- `lockDown`
- `unlock`

Core Invariants:

INV 1: Non-unanimous Safe transactions should not be executable unless they have first been queued through `enqueue()` and the configured timelock has elapsed.

INV 2: A queued transaction should not inherit future delay changes. Updating the SafeGuard delay should only affect transactions queued after the update.

INV 3: Unanimous owner approval should be an immediate execution path and shouldn't require the transaction to pass through the timelock.

INV 4: Safe transactions should not be able to directly call `checkTransaction()` or `checkAfterExecution()` on the SafeGuard outside the Safe's normal guarded execution flow.

INV 5: Any Safe transaction that calls `unlock()` should require unanimous approval from all owners.

INV 6: The only permitted `delegatecall` target should be the expected `MultiSendCallOnly` contract.

INV 7: User-selected calls inside a `MultiSendCallOnly` `delegatecall` batch should be interleaved by SafeGuard `check()` calls.

INV 8: Any individual Safe owner should be able to trigger lockdown or cancel.

INV 9: While the SafeGuard is locked down, ordinary queued or threshold-approved transactions should not be executable.

INV 10: A transaction should only be cancelable while it is still inside the timelock window.

INV 11: Once a transaction hash has been canceled, that exact transaction hash should not be queueable again.

INV 12: A caller should not be able to cancel a queued transaction unless they have already pre-approved their own resignation transaction.

INV 13: A caller should not be able to cancel the transaction hash corresponding to their own resignation.

INV 14: A locking owner should not be able to put the Safe into lockdown without also committing to an unlock path.

INV 15: After each transaction, the Safe must still use the expected implementation, must not have any modules installed, must retain the expected fallback handler, and must not have the SafeGuard itself as an owner.

INV 16: After each guarded transaction, the Safe must maintain the minimum owner count, the minimum threshold, and enough owners outside the threshold.

Issue_01	A malicious owner can repeatedly re-lock the Safe before their removal is executed
Severity	Medium
Description	The <code>lockDown()</code> function requires the caller to pre-approve an <code>unlock()</code> transaction at the current Safe nonce. This ensures that the remaining owners can recover from lockdown even if the caller is malicious, because the caller has already provided the approval needed for <code>unlock()</code>. This preapproval only applies to a standalone <code>unlock()</code> transaction hash. It does not preapprove a transaction that both calls <code>unlock()</code> and removes the <code>lockDown()</code> caller. It's worth noting that there is a separate requirement to extend a pre-approved removal call to the next nonce if the caller has already pre-approved their current-nonce removal. This only applies when the caller has previously created such a removal commitment in the <code>cancel()</code> flow, but it does not apply to a caller who only uses <code>lockDown()</code>. If an owner is malicious and using <code>lockDown()</code> to freeze or stall the Safe, the desired recovery flow is likely to unlock the Safe and remove that owner. However, the malicious owner would presumably not cooperate in signing an atomic <code>unlock() + removeOwner()</code> transaction (which would require unanimous signatures), and their standalone <code>unlock()</code> preapproval cannot be used with such a batched transaction, because it'd be a different transaction hash. As a result, the remaining owners would need to execute the pre-approved standalone <code>unlock()</code> transaction first, then remove the malicious owner in a separate transaction. This creates a potential race condition: if the malicious owner can submit another <code>lockDown()</code> transaction after the Safe is unlocked but before their removal is executed, they can re-lock the Safe and force the recovery process to repeat. This is worsened by the fact that <code>enqueue()</code> cannot be called during a lockdown. Therefore, if the malicious owner's removal was not

	already queued before the lockdown, the remaining owners must first execute <code>unlock()</code>, then queue the removal transaction, wait the full timelock period, and only then execute the removal. This gives the malicious <code>lockDown()</code> caller a wide window of time to call <code>lockDown()</code> again before they can be removed. It's worth noting that the malicious owner is not allowed to <code>cancel()</code> this removal, but cancelling is not required in what's being described here. In summary, if a malicious owner starts calling <code>lockDown()</code>, they only commit to a standalone <code>unlock()</code> transaction, not to their own removal. No new transaction can be queued through <code>enqueue()</code> until the <code>unlock()</code> is executed, but after that, the entire timelock window is available for the malicious owner to call <code>lockDown()</code> again and continue freezing the Safe.
Recommendations	Consider this behavior, and consider requiring a batched unlock and resignation preapproval on every <code>lockDown()</code>, instead of just a standalone <code>unlock()</code> call. Simultaneously, consider allowing transactions to be enqueued during a lockdown. Note that one potential workaround of this issue is for honest owners to immediately enqueue multiple removals of the malicious owner at several future nonces once the Safe is unlocked. This can let the removals wait through the timelock while the malicious owner re-locks the Safe, so that a later standalone <code>unlock()</code> can be manually batched with an already-ready removal.
Comments / Resolution	Acknowledged. The client has confirmed that the workaround mentioned above is the intended response for this scenario. The honest owners would privately gather the signatures needed to both unlock the Safe and queue removal transactions for the malicious owner at the two next nonces, then submit them together. From there, the removal either succeeds after the timelock without another lockdown, or, if the malicious owner calls <code>lockDown()</code> again, the honest owners can wait until the queued removal at the future nonce is ready and then atomically unlock the Safe and execute the removal, for example using a private RPC service. Also note that disallowing enqueue while in lockdown is deliberate because it

	prevents key-compromised owners from spamming malicious transactions into the queue.
--	--

Issue_02	Owner changes during an <code>unlock()</code> can invalidate a pre-approved resignation from <code>cancel()</code>
Severity	Medium
Description	When the SafeGuard is in lockdown mode, <code>cancel()</code> checks the caller has provided a pre-approved resignation at the next nonce rather than the current nonce. This behavior appears to be intended to account for the fact that, while locked down, the current Safe nonce will be consumed by the future <code>unlock()</code> transaction. In other words, if the current nonce is n, then <code>cancel()</code> requires the caller to have preapproved their own resignation at nonce $n + 1$, so that the resignation can be executed after the Safe is unlocked. However, the resignation hash does not only depend on the nonce. It also depends on the current owner linked-list ordering, because the signed <code>removeOwner()</code> call must include the owner's current <code>prevOwner</code>, and that predecessor must still be correct when the resignation is executed. This means the preapproved resignation is only useful if the owner ordering remains compatible with it after <code>unlock()</code> executes. If the unlock transaction also changes the Safe owner ordering (which may be possible if the <code>unlock()</code> happens during a multicall), the <code>prevOwner</code> embedded in the preapproved resignation may no longer be correct. This weakens the intended anti-griefing properties of <code>cancel()</code>.
Recommendations	This behavior seems difficult to change since it's unknown at the time of the cancel what the <code>unlock()</code> transaction will do. Keep this behavior in mind when doing <code>unlock()</code> within multicalls.

Comments / Resolution	Acknowledged.
-----------------------	---------------

Issue_03	Minimum owner-count postconditions block eviction of a malicious owner at the guard floor (allowing griefing and capture via cancel / timelock)
Severity	Medium
Description	SafeGuard enforces floor values of <code>MINIMUM_OWNERS</code> = 4 and <code>MINIMUM_THRESHOLD</code> = 2, and after every guarded execution, <code>checkAfterExecution</code> requires the Safe to satisfy those invariants, including <code>ownerCount</code> $\geq$ 4 and <code>ownerCount</code> - <code>threshold</code> $\geq$ 2. Any <code>removeOwner</code> transaction that would leave the Safe below these bounds will therefore fail. This can be problematic when the owner being removed is malicious and their removal is necessary for recovery/anti-griefing. For example, in a 2/4 Safe, the anti-griefing protection in <code>cancel()</code> does not work correctly. Even though the cancelling owner pre-approves their own removal, that removal would leave the Safe as 2/3, which violates the guard's postconditions. As a result, the pre-approved removal cannot actually be executed.
Recommendations	Consider raising the guard's minimum owner count and/or <code>threshold</code> so that the intended "normal" operating point is strictly above the floor enforced in <code>checkAfterExecution</code> , leaving headroom to remove one owner without violating postconditions.
Comments / Resolution	Acknowledged, and will document that the initial configuration of the Safe should be 2/5 at minimum.

Issue_04	Gas refunds create a reentrancy window before SafeGuard invariant checks
Severity	Medium
Description	The <code>checkAfterExecution()</code> function is intended to enforce SafeGuard's post-execution invariants. The codebase implicitly assumes that no attacker-controlled code can execute after the Safe tx but before these invariant checks run. This assumption is important because if one of the invariants is violated, it can often be weaponized immediately to take over the Safe, then clean up before <code>checkAfterExecution()</code> observes the broken state. Gas refunds break this assumption. The refund payment (via <code>handlePayment()</code> in the Safe) is performed after the Safe tx has executed, but before <code>checkAfterExecution()</code> is called. Since this payment can transfer control flow to attacker-controlled code, it creates a reentrancy window between the Safe tx and the final invariant checks. For example, if a module is installed during the Safe tx, a malicious owner set could regain control flow during the refund payment, use the newly installed module to modify the Safe arbitrarily, and then restore the expected state before <code>checkAfterExecution()</code> runs. Similar behavior exists if the SafeGuard is removed during the Safe tx. During the refund payment, the Safe can be reentered while the guard is already removed, allowing the Safe to be used without any guard enforcement. This all occurs before the guard-removal transaction has reached its own <code>checkAfterExecution()</code> call. These are two examples, but this appears to be a broader issue: that any invariant enforced by <code>checkAfterExecution()</code> can be temporarily violated during the refund payment and then restored before the final check observes it. Other relevant invariants that are checked in <code>checkAfterExecution()</code> include: - <code>safe.masterCopy() == SINGLETON</code>

	- <code>safe.getFallback() == _FALLBACK</code> - <code>safe.isOwner(address(this)) == false</code> - the owner count and threshold bounds (side note: this one is debatably not an issue, since the multisend logic already allows them to be temporarily invalid until the final <code>checkAfterExecution()</code> check)
Recommendations	Consider disallowing gas refunds for any transaction that can affect any of these invariant checks. It may be difficult to blacklist all different types of txs that can affect <code>checkAfterExecution()</code> invariants. It may therefore be worth considering disallowing all gas refund functionality out of caution.
Comments / Resolution	Fixed. ERC20 gas sponsorship is now disallowed for transactions that call <code>setGuard()</code>, <code>enableModule()</code>, or <code>setFallbackHandler()</code> on the Safe, either directly or through <code>MultiSendCallOnly</code>. Native-token sponsorship remains allowed because the currently supported Safe versions use <code>.send()</code> for native refunds, limiting the receiver to 2300 gas. This assumption should be revisited if SafeGuard support is expanded to Safe v1.5.0 or newer, where native refunds no longer use <code>.send()</code>. The remaining <code>checkAfterExecution()</code> invariants did not have additional changes for the following reasoning: - <code>safe.masterCopy() == SINGLETON</code>: changing the implementation requires a <code>delegatecall</code>, and arbitrary <code>delegatecalls</code> are already prohibited. - <code>safe.isOwner(address(this)) == false</code>: this invariant prevents the Safe from treating a call made by the SafeGuard as owner authorization during signature validation. Temporarily making the SafeGuard an owner is not abusable by itself in the <code>handlePayment()</code> reentrancy window. This would only become relevant if it could be chained with some separate issue that causes the SafeGuard itself to call back into the Safe while the invariant is temporarily violated. - Owner count and threshold bounds: these values can already be temporarily violated during a <code>MultiSendCallOnly</code> flow, so this is not new behavior.

Issue_05	<code>cancel</code> can be used by an owner to cancel their own enqueued resignation when other txs are queued
Severity	Medium
Description	<code>Cancel</code> is designed to permit any owner to cancel any queued transaction, except their own resignation. However, this can be bypassed if there are already any transactions in the queue, by nonce bumping. Say there is some transaction in the queue, with nonce 100 and a <code>removeOwner</code> tx with nonce 101, which kicks out the malicious owner. The malicious owner can then call <code>cancel</code> on this <code>removeOwner</code> transaction, bypassing the check. This is because, due to the other enqueued transaction, the current nonce of the safe [100] doesn't match the <code>removeOwner</code> tx nonce [101]. So the <code>resignHash</code> will not match the queued <code>txHash</code>. <pre> bytes32 resignHash = _removeOwnerTxHash(_safe, _safe.getPrevOwner(msg.sender), msg.sender, _getThresholdAfterResign(_safe), nonce); if (resignHash == txHash) { revert CannotCancelOwnResignation(txHash); } </pre> So the malicious owner can cancel their own resignation transaction, holding the safe hostage.
Recommendations	Consider storing more details about the transaction in <code>enqueue</code> and not just the tx hash. That way, resignation transactions can be detected more easily and forbidden.
Comments / Resolution	Fixed as recommended. When a direct <code>removeOwner()</code> call is queued, <code>enqueue()</code> now records the owner being removed in the transaction's stored data. <code>cancel()</code> then checks this data and reverts if the caller is an owner being removed.

Issue_06	Owners can cancel multicall transactions that include their own removal
Severity	Low
Description	The self-removal check in <code>cancel()</code> only covers a direct <code>removeOwner</code> call for the cancelling owner at the current Safe nonce. It does not detect transactions where the caller's removal is performed inside a multicall. A multicall that includes <code>removeOwner</code> has a different Safe transaction hash than a standalone <code>removeOwner</code> call. So, owners can cancel multicalls that are removing them, even at the current nonce. This may affect the anti-griefing logic and make it more difficult to remove a malicious owner.
Recommendations	Consider this behavior together with the related issue regarding cancelling future nonce removals. One possible fix is to refactor <code>cancel()</code> so that it takes the full transaction being cancelled as input and derives the <code>txHash</code> internally. This would allow the contract to inspect the cancelled transaction and reject more types of cancellations of the caller's own removal.
Comments / Resolution	Acknowledged. It's worth noting that if <code>removeOwner()</code> calls inside multicalls were also made uncancellable, malicious owners could bundle a harmful action with an owner-removal call and make the entire multicall impossible for that owner to cancel, which is dangerous.

Issue_07	<code>cancel()</code> and <code>lockdown()</code> don't check reentrancy guards
Severity	Low
Description	The <code>cancel()</code> and <code>lockDown()</code> functions do not check the <code>SafeGuard</code>'s reentrancy guard, meaning they can potentially be called while a Safe transaction is mid-execution inside the <code>checkTransaction()</code> / <code>checkAfterExecution()</code> flow. For example, this may be possible if a Safe call gives up control flow during a <code>MultiSendCallOnly</code> batch. This can make the state observed by these functions differ from the final state of the Safe transaction. For example, <code>cancel()</code> computes the required resignation hash based on the Safe nonce and whether <code>lockedDownBy</code> is currently set. During an in-flight transaction, the Safe nonce has already been incremented, and <code>lockedDownBy</code> may still be set even if the transaction is about to call <code>unlock()</code> later in the same batch. In that case, <code>cancel()</code> may check against the wrong resignation nonce. Similarly, a reentrant call to <code>lockDown()</code> computes the <code>getPrevOwner()</code>, however the same Safe transaction may later modify the owner set before completing, which makes this logic difficult to reason about and potentially incorrect in this intermediate state.
Recommendations	Consider adding a reentrancy check to <code>cancel()</code> and <code>lockDown()</code> so they revert while <code>_reentrancyGuard</code> is set.
Comments / Resolution	Fixed by adding a revert if the reentrancy guard is active in <code>cancel()</code> , <code>lockDown()</code> , and also <code>enqueue()</code> .

Issue_08	Predicted addresses may not be created yet
Severity	Low
Description	The SafeGuard contract predicts the Safe singleton, fallback handler, and MultiSendCallOnly addresses using the same factory that is deploying the SafeGuard. However, there is no guarantee that the factory has already deployed those three contracts at the predicted addresses. If any predicted address is empty, the SafeGuard can end up pointing to an address with no code. This is especially relevant for the MultiSendCallOnly contract, since it is not configured in the Safe itself and therefore may go unnoticed if it has not yet been deployed by the factory.
Recommendations	In the constructor, consider adding a check that the predicted singleton, fallback handler, and MultiSendCallOnly addresses contain code.
Comments / Resolution	Fixed by adding code-length checks in <code>_constructorChecks()</code> for each of the three predicted addresses.

Issue_09	approvedHashes ABI return-type mismatch
Severity	Low
Description	Gnosis Safe stores approvals in <code>mapping[address => mapping[bytes32 => uint256]]</code> public <code>approvedHashes</code>, so the auto-generated getter returns <code>uint256</code> values, typically 0 or 1. In this repo, <code>ISafeMinimal</code> instead declares <code>approvedHashes(address, bytes32)</code> returns <code>[bool]</code>, so <code>SafeGuard</code> decodes the return value as a boolean, for example, in <code>_requirePreApproved</code>, which does <code>if (!_safe.approvedHashes(msg.sender, txHash))</code>. This technically works for the current Safe implementation because ABI-encoded false/true return values would be identical to <code>uint256[0]/uint256[1]</code>. However, the interface type is incorrect, and relying on this implicit compatibility is dangerous, especially if values other than 0 or 1 are possible in future versions.
Recommendations	Change <code>ISafeMinimal</code> to <code>approvedHashes(...)</code> returns <code>[uint256]</code>, and replace boolean uses with explicit checks, e.g. <pre><code>_safe.approvedHashes(owner, h) != 0</code></pre>
Comments / Resolution	Acknowledged due to the behavior working correctly for v1.3.0 and v1.4.1 Safes.

Issue_10	Safe 1.1.1 proxy codehash is whitelisted, but Safe 1.1.1 has no Guard support
Severity	Low
Description	In SafeGuard.sol, _isSupportedProxyCodeHash whitelists three Safe proxy codehashes: <pre>bytes32 private constant _SAFE_PROXY_1_1_CODEHASH = 0xaea7d4252f6245f301e540cfbee27d3a88de543af8e49c5c62405d 5499fab7e5; bytes32 private constant _SAFE_PROXY_1_3_CODEHASH = 0xb89c1b3bdf2cf8827818646bce9a8f6e372885f8c55e5c07acbd30 7cb133b000; bytes32 private constant _SAFE_PROXY_1_4_CODEHASH = 0xd7d408ebcd99b2b70be43e20253d6d92a8ea8fab29bd3be7f55b1 0032331fb4c;</pre> _constructorChecks accepts the Safe if its codehash matches any of these. However, Gnosis Safe 1.1.1 does not implement the GuardManager interface - there is no setGuard/getGuard storage slot or checkTransaction/checkAfterExecution hook in 1.1.1. It's unclear if this is intended. It's theoretically possible that Proxy version 1.1.1 could be used with a more recent singleton version like 1.3, in which case such a setup could potentially work.
Recommendations	Consider if v1.1.1 should be supported, and remove _SAFE_PROXY_1_1_CODEHASH from the whitelist if not.
Comments / Resolution	Acknowledged, the intended use case is a v1.1.1 proxy pointing to a v1.3.0 or v1.4.1 singleton.

Issue_11	Enqueue accepts historical signatures
Severity	Low
Description	The Enqueue function does not check the current nonce of the safe, meaning anyone can call it and queue up a transaction from the past, as long as they have the required signatures. If this SafeGuard is set up on an existing Safe wallet, users can queue transactions from the past by using the signatures available on-chain. For example, if the Safe wallet passed a tx with nonce 100 with 3 signatures, and then SafeGuard was added at nonce 200, anyone can queue up the old transaction of nonce 100 again using the 3 signatures already available on-chain. While this transaction cannot be actually executed, events are still emitted during the enqueue process, polluting the log.
Recommendations	Consider only allowing enqueue for transactions whose nonce is $\geq$ the safe.nonce .
Comments / Resolution	Fixed as recommended.

Issue_12	Removability commitment can be evaded by triggering a Safe nonce bump immediately afterwards
Severity	Low
Description	<code>cancel</code> requires the caller to have pre-approved their own resignation hash before they are allowed to cancel any queued transaction. <pre> bytes32 resignHash = _removeOwnerTxHash(_safe, _safe.getPrevOwner(msg.sender), msg.sender, _getThresholdAfterResign(_safe), nonce); ... _requirePreApproved(_safe, resignHash); </pre> The intent is anti-griefing: an owner who cancels a queued tx commits to being removable at the same nonce. However, this can be bypassed by nonce-bumping if there are other transactions in the queue already. Say the safe has nonce 100, and there are 2 transactions in the queue with nonce 100 and 101, and the nonce 100 transaction is close to being executable. The malicious owner can then cancel the nonce 101 transaction by approving a resignation with the current nonce (100). But once they execute the pending nonce 100 transaction present in the queue, the wallet nonce goes to 101, and their approved resignation becomes meaningless, since it is not executable anymore. So the user is able to cancel a transaction without risking their own removal by manipulating the nonce. The same issue is also applicable to the lockdown function, which also requires a similar commitment, which can be bypassed by executing a transaction once unlocked again.

Recommendations	Consider implementing a way to prevent nonces of queued transactions to be skipped during signing/pre-approval.
Comments / Resolution	Acknowledged.

Issue_13	<code>check()</code> multicall validation allows trailing calldata
Severity	Informational
Description	When validating <code>MultiSendCallOnly</code> batches, <code>_checkDelegateCall()</code> requires every other sub-call to call <code>check()</code> on the <code>SafeGuard</code>. This validation checks that the sub-call calldata is at least 4 bytes long and that the first 4 bytes match the <code>check()</code> selector. Notably, this check allows the <code>check()</code> sub-call calldata to be longer than 4 bytes, even though a normal <code>check()</code> call should only contain the function selector. This should not cause any practical issue, as Solidity will ignore the trailing calldata rather than reverting. However, requiring the calldata length to be exactly 4 bytes would more precisely enforce the intended <code>check()</code> call.
Recommendations	Consider requiring <code>multicallData.length == 4</code> for the interleaved <code>check()</code> calls.
Comments / Resolution	Acknowledged.

Issue_14	Inaccurate comment for <code>_checkAfterExecutionReturnBool()</code>
Severity	Informational
Description	The comment above <code>_checkAfterExecutionReturnBool()</code> states that it performs exactly the same checks as <code>_checkAfterExecution()</code>, except that it returns false instead of reverting. This is not strictly accurate. <code>_checkAfterExecutionReturnBool()</code> also includes the owner and threshold checks, while <code>_checkAfterExecution()</code> does not. However this behavior appears intentional, since <code>checkAfterExecution()</code> performs the owner and threshold checks immediately after calling <code>_checkAfterExecution()</code> anyway.
Recommendations	Consider updating the comment to reflect this difference between <code>_checkAfterExecutionReturnBool()</code> and <code>_checkAfterExecution()</code> .
Comments / Resolution	Fixed as recommended.

Issue_15	Signature checks can differ between enqueue and execution
Severity	Informational
Description	The <code>enqueue()</code> function validates that the transaction has sufficient Safe signatures at the time it is added to the timelock. However, the Safe owner set or threshold may change before the queued transaction is later executed. This isn't an issue since the signatures are checked again in the Safe during execution (and also checked once more in <code>checkTransaction()</code>, to see if the approval is unanimous).
Recommendations	Keep this behavior in mind when reasoning about queued transactions, especially around owner or threshold changes that occur during the timelock period.
Comments / Resolution	Acknowledged.

Issue_16	Reducing the owner count can cause a previously-enqueued transaction to bypass the timelock
Severity	Informational
Description	In <code>checkTransaction</code>, the unanimity check uses the current owner count at execution time: <pre>try _safe.checkNSignatures(txHash, txHashData, signatures, _safe.ownerCount() // <-- evaluated NOW, not at enqueue time){return;} catch {...}</pre> <code>enqueue</code> requires only <code>safe.checkSignatures</code> (current threshold), and the signatures supplied at enqueue time are kept off-chain by the submitter and re-supplied at execution. So if a transaction was signed by K out of K+1 signers, and then submitted in the queue, it is still expected to go through the entire timelock. However, if the last signer (the one who did not sign it) is removed during this timelock period, the transaction can be passed immediately as a unanimous transaction, bypassing the timelock. Thus, removing signers can change the state of already queued transactions.
Recommendations	Consider documenting and acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_17	Transaction cancellation is permanent
Severity	Informational
Description	The <code>cancel()</code> function does not simply remove a transaction from timelock storage. Instead, it permanently marks the transaction hash as cancelled by setting its timelock to the maximum possible value. As a result, the same transaction hash can never be queued again, since the guard treats any non-zero timelock entry as already queued.
Recommendations	Keep this behavior in mind, and consider documenting it further.
Comments / Resolution	Acknowledged.

Issue_18	Future-nonce transactions can become non-cancellable before execution
Severity	Informational
Description	The <code>enqueue()</code> function accepts a caller-provided nonce and does not require it to match the Safe's current nonce. This allows owners to queue a transaction for a future nonce, such as <code>currentNonce + 10</code>. On the other hand, <code>cancel()</code> only works before the queued transaction's timelock expires. As a result, a future-nonce transaction can remain queued until its timelock expires, while still being impossible to execute because earlier Safe nonces have not yet been consumed. Once the timelock expires, single-owner cancellation is no longer available. If the Safe later reaches that nonce, the transaction can still be executed. This may affect the game theory around malicious or compromised owners, since a transaction can become non-cancellable before it is actually executable.
Recommendations	Consider the consequences of allowing cancellation even after the timelock has passed. If there are no meaningful downsides to this behavior, consider supporting cancellation beyond the timelock expiry.
Comments / Resolution	Acknowledged.

Issue_19	Guard assumes one action per Safe call
Severity	Informational
Description	An implicit assumption of the multicall logic in the codebase is that a single Safe call cannot perform multiple independent actions. This matters because the guard relies on inspecting each multicall action and requiring <code>check()</code> calls between them. So, for example, if one Safe call could add a module and then call into that module as part of the same action, it could bypass this system. Based on the current Safe code, it is indeed not possible to batch multiple actions in one call, so this does not appear to be an issue.
Recommendations	This finding has been provided for informational purposes, no changes are needed. Consider documenting this assumption and keeping it in mind when supporting future Safe versions.
Comments / Resolution	Acknowledged.

Issue_20	Newer MultiSend contracts have different <code>address[0]</code> behavior
Severity	Informational
Description	Some newer versions of Safe's MultiSendCallOnly contract special-case subcalls with target <code>address[0]</code> and replace the target with <code>address[this]</code>. Since MultiSendCallOnly is executed through delegatecall, <code>address[this]</code> is the Safe. This is not the case for the versions of MultiSendCallOnly referenced in the codebase, so there is no issue. Also, the Safe address itself isn't a special case in the codebase anyway.
Recommendations	This finding has been provided for informational purposes, no changes are needed. Keep this behavior in mind for future versions of the codebase.
Comments / Resolution	Acknowledged.

Issue_21	Signer independence is not enforced by Safe owner address validation
Severity	Informational
Description	The SafeGuard threat model describes recovery and quorum assumptions in terms of keys, signers, and keyholders. However, the implementation relies on Safe's normal signature validation, which counts distinct Safe owner addresses rather than independent underlying controllers. As a result, multiple owner addresses controlled by the same keyholder can still satisfy quorum or unanimous-signature checks. For example, an EOA owner and an ERC-1271 contract owner controlled by that same EOA are treated as two distinct Safe owners. This is not a bypass of Safe's rules, but it means the guard's guarantees are address-level unless signer independence is enforced externally.
Recommendations	This finding has been provided for informational purposes, no changes are needed. Keep this behavior in mind for future versions of the codebase.
Comments / Resolution	Acknowledged.